

UNIVERSIDAD AUTÓNOMA DEL PERÚ

FACULTAD DE INGENIERÍA Y ARQUITECTURA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

EVALUACIÓN PARCIAL	PROGRAMACIÓN SEGURA — DD281
SEMESTRE	2026-1
DOCENTE	Mg. Ruben Quispe Llachtarimay
DURACIÓN	300 minutos (5 horas) Fecha: _____

CÓDIGO DEL ESTUDIANTE:	_____	NÚMERO DE CLASE:	_____
APELLIDOS Y NOMBRES:	_____	SECCIÓN:	_____

PROYECTO: SITIO WEB CORPORATIVO

UQ AI SOLUTION COMPANY

CONTEXTO DEL PROYECTO

UQ AI SOLUTION COMPANY es una empresa peruana emergente especializada en Inteligencia Artificial. Tu misión es **diseñar, implementar y desplegar** un sitio web corporativo moderno y seguro, similar a *deeplearning.ai* o *landing.ai*, aplicando los principios de **Programación Segura** vistos en clase.

DIVISIÓN	SERVICIOS
 UQ AI Solutions	Agentes IA · Chatbots empresariales · Automatización · MYPES · Salud & Educación · Big Data · Programación Segura · DevSecOps
 UQ AI Academy	Cursos de IA · Machine Learning · RAG · LLM · Agentes inteligentes · Programación segura · Big data · Cloud computing
 UQ AI Lab	Prototipos · Demos · Investigación aplicada · Proyectos con estudiantes · Comunidad de IA

 **NOTA:** Puedes usar IA Generativa (Claude, GitHub Copilot, ChatGPT, v0.dev) para generar código. Lo que se evalúa es que el código **funcione correctamente**, aplique **seguridad real** y tú puedas explicar cada parte.

STACK TECNOLÓGICO OBLIGATORIO

CAPA	TECNOLOGÍA	PROPÓSITO
Frontend	Next.js 14 + TypeScript + Tailwind CSS	Framework React moderno con SSR/SSG

Backend	Java Spring Boot 3.x + Spring Security	API REST robusta con seguridad integrada
Base de datos	H2 (en memoria) + JPA/Hibernate	BD embebida, sin instalación adicional
Autenticación	JWT + bcrypt (BCryptPasswordEncoder)	Programación Segura: hashing + tokens
Deploy Frontend	Vercel (gratuito)	HTTPS automático, CDN global
Deploy Backend	Railway.app o Render.com (gratuito)	Hosting Java gratuito
Repositorio	GitHub	Control de versiones y evidencia de commits
Asistente IA	Claude / GitHub Copilot / ChatGPT / v0.dev	Acelerar el desarrollo del proyecto

DESARROLLO

PREGUNTA 1 — Sitio Web Frontend: Landing Page UQ AI Solution (6 puntos)

- **1.1 Sección Hero:** Nombre de la empresa, tagline "Inteligencia Artificial para el Perú y el Mundo", botones CTA ("Explorar Servicios", "Contactar") e imagen/animación de fondo. [1 pto]
- **1.2 Sección UQ AI Solutions:** Tarjetas de servicio con íconos, título y descripción. Mínimo 6 servicios visibles (agentes IA, chatbots, automatización, MYPES, salud/educación, Big Data). [1 pto]
- **1.3 UQ AI Academy y UQ AI Lab:** Listado visual de cursos disponibles y proyectos del laboratorio con diseño de cards. [1 pto]
- **1.4 Formulario de Contacto (Lead Form):** Campos: Nombre, Email, Empresa, Teléfono, Mensaje. Al enviar, guarda el lead en el backend vía API REST. [1 pto]
- **1.5 Navbar responsivo:** Logo, menú de navegación (Servicios, Academia, Lab, Contacto, Iniciar Sesión) y versión hamburguesa en mobile. [1 pto]
- **1.6 Footer y diseño general:** Footer con info empresa, redes sociales, año y aviso legal. Diseño profesional comparable a deeplearning.ai o landing.ai (paleta coherente, tipografía moderna). [1 pto]

📺 Enviar video de evidencia (máx. 3 min) mostrando el sitio en desktop y mobile. Link público (YouTube / Drive / Loom).

PREGUNTA 2 — Backend Seguro: Spring Boot + H2 + REST API (5 puntos)

- **2.1 Proyecto Spring Boot (0.5 pt):** Dependencias: Spring Web, Spring Security, Spring Data JPA, H2, Lombok, Validation, jjwt. Nombre: T_Apellido_Nombre_backend.
- **2.2 Entidad Usuario + bcrypt (1 pt):** Atributos: id, nombre, apellidos, email, password (hash bcrypt), rol (ADMIN/USER), area. La contraseña NUNCA se almacena en texto plano — usar BCryptPasswordEncoder(12).
- **2.3 Entidad Lead — formulario contacto (0.5 pt):** Atributos: id, nombre, email, empresa, telefono, mensaje, fechaRegistro (@CreationTimestamp).
- **2.4 Endpoints REST (1.5 pt):** POST /api/auth/register · POST /api/auth/login (retorna JWT) · GET /api/usuarios (solo ADMIN) · GET /api/usuarios/{id} (ADMIN o mismo USER) · POST /api/leads (público) · GET /api/leads (solo ADMIN).
- **2.5 Spring Security + JWT (1 pt):** CORS para dominio frontend, CSRF desactivado (JWT stateless), filtro JWT, endpoints /api/auth/** y POST /api/leads públicos.

- **2.6 H2 configurado (0.5 pt):** Consola H2 en /h2-console, BD: BD_apellido_nombre_T1. Verificar con Postman o curl los 6 endpoints.

PREGUNTA 3 — Login Seguro y Panel de Administración (5 puntos)

- **3.1 Página /login con validaciones de seguridad (1 pt):** Formulario Email + Contraseña, mensaje de error genérico ("Credenciales incorrectas" — sin revelar qué campo falló), loading state, bloqueo del botón 30 segundos tras 3 intentos fallidos.
- **3.2 JWT en Cookie HttpOnly — NO en localStorage (1 pt):** Al recibir el JWT del backend, guardarlo en cookie HttpOnly=true, Secure=true (producción), SameSite=Strict vía Route Handler de Next.js. Comentar en el código por qué NO se usa localStorage.
- **3.3 Rutas protegidas con middleware.ts (1 pt):** Si el usuario accede a /dashboard o /admin sin cookie JWT válida → redirigir a /login. Si ya está logueado e intenta ir a /login → redirigir a /dashboard.
- **3.4 Dashboard con RBAC (1 pt):** ADMIN: tabla completa de leads (GET /api/leads con JWT en Authorization header). USER: solo ve su propio perfil. Distinción visual clara de rol.
- **3.5 Logout seguro (1 pt):** Al hacer clic en "Cerrar Sesión", eliminar la cookie en el servidor (Max-Age=0 vía Route Handler) Y limpiar estado local. Redirigir a /login.

PREGUNTA 4 — Despliegue y Repositorio GitHub (2 puntos)

- **4.1 GitHub con mínimo 3 commits significativos (1 pt):** Los commits deben reflejar el avance: setup inicial → implementación features → deploy. No vale un solo commit con todo el código. Enviar URL del repositorio público.
- **4.2 Deploy Vercel (frontend) + Railway/Render (backend) (1 pt):** Frontend en Vercel: conectar repo GitHub, variable NEXT_PUBLIC_API_URL apuntando al backend. Backend en Railway.app o Render.com. Enviar ambas URLs públicas operativas.

PREGUNTA 5 — Video Evidencia de Funcionamiento (2 puntos)

Grabar un video de máximo 5 minutos mostrando los siguientes 6 puntos:

1. Sitio web cargando desde la URL pública de Vercel (dominio .vercel.app)
2. Formulario de contacto enviando un lead → verificar en panel admin que llegó
3. Login con usuario ADMIN → acceso al dashboard con tabla completa de leads
4. Login con usuario USER → solo ve su propio perfil (RBAC funcionando)
5. Intento de acceder a /dashboard sin login → redirige automáticamente a /login
6. Logout → cookie eliminada (mostrar en DevTools > Application > Cookies que desapareció)

 **El link del video debe ser PÚBLICO (YouTube no listado, Google Drive con acceso o Loom).**

RÚBRICA DE CALIFICACIÓN

ÍTEM	DESCRIPCIÓN	COMPLETO (100%)	PARCIAL (50%)	NO RESPONDE
Pregunta 1	Landing Page Next.js (Hero + Services + Academy + Lab + Form + Footer)	6	3	0
Pregunta 2	Backend Spring Boot + H2 + 6 Endpoints REST + Spring Security	5	2.5	0
Pregunta 3	Login Seguro + JWT Cookie HttpOnly + Rutas Protegidas + RBAC + Logout	5	2.5	0
Pregunta 4	GitHub (≥3 commits) + Deploy Vercel + Railway/Render	2	1	0
Pregunta 5	Video Evidencia 5 min (6 puntos de demostración)	2	1	0
TOTAL: 20				

Detalle de Rúbrica — Pregunta 1: Landing Page (6 puntos)

SUB-ÍTEM	COMPLETO (1 pt)	PARCIAL (0.5 pt)	NO RESPONDE (0 pt)
1.1 Hero (1 pt)	Hero completo, animación/imagen, 2 CTA funcionando, tagline visible	Solo texto sin diseño, sin CTA	Sin sección Hero
1.2 Services Cards (1 pt)	6+ servicios con íconos y descripción completa	3-5 servicios o sin íconos	Sin sección Services
1.3 Academy + Lab (1 pt)	Ambas secciones con cards completas y contenido real	Solo una sección o diseño incompleto	Sin las secciones
1.4 Lead Form (1 pt)	Formulario envía datos al backend, confirmación visual	Formulario sin conexión al backend	Sin formulario de contacto
1.5 Navbar Responsive (1 pt)	Funciona en desktop y mobile (menú hamburguesa)	Solo desktop o roto en mobile	Sin navbar
1.6 Footer + Diseño (1 pt)	Profesional, comparable a deeplearning.ai/landing.ai	Diseño básico o incompleto	Sin footer o diseño muy pobre

Detalle de Rúbrica — Pregunta 2: Backend (5 puntos)

SUB-ÍTEM	COMPLETO	PARCIAL	NO RESPONDE
2.1 Setup (0.5 pt)	Proyecto corre sin errores	Corre con warnings menores	No compila
2.2 Usuario + bcrypt (1 pt)	Hash bcrypt aplicado, nunca texto plano	Hash implementado con errores	Contraseñas en texto plano
2.3 Entidad Lead (0.5 pt)	Todos los campos y @CreationTimestamp	Faltan campos opcionales	Sin entidad Lead
2.4 Endpoints REST (1.5 pt)	Los 6 endpoints funcionan (verificado con Postman)	3-5 endpoints funcionan	0-2 endpoints
2.5 Spring Security + JWT (1 pt)	JWT válido, CORS configurado, filtro OK	JWT implementado con errores	Sin configuración de seguridad
2.6 H2 configurado (0.5 pt)	Consola H2 accesible, BD con nombre correcto	H2 funciona pero mal configurada	Sin H2

Detalle de Rúbrica — Pregunta 3: Login Seguro (5 puntos)

SUB-ÍTEM	COMPLETO (1 pt)	PARCIAL (0.5 pt)	NO RESPONDE (0 pt)
3.1 Página Login (1 pt)	Mensaje genérico, loading state, bloqueo 3 intentos	Login funciona sin validaciones de seguridad	Sin página de login
3.2 Cookie HttpOnly (1 pt)	JWT en HttpOnly + Secure + SameSite=Strict	JWT en localStorage o sessionStorage (INSEGURO)	Sin almacenamiento de token
3.3 Rutas Protegidas (1 pt)	Middleware redirige correctamente (ambas rutas)	Protección parcial o bypaseable	Sin protección de rutas
3.4 Dashboard + RBAC (1 pt)	Admin ve leads, User ve solo su perfil	Solo un rol implementado	Sin dashboard
3.5 Logout Seguro (1 pt)	Cookie eliminada en servidor, estado limpio, redirige	Solo limpia estado en cliente	Sin logout o inoperativo

GUÍA DE DESARROLLO PASO A PASO

 Tiempo estimado con IA Generativa: **3 horas**

TIEMPO	ETAPA	ACCIONES
HORA 1	Setup y Estructura	• <code>npx create-next-app@14 uq-ai-frontend --typescript --tailwind --app</code> • Crear proyecto Spring Boot en <code>start.spring.io</code> (Web + Security + JPA + H2 + Lombok) • Agregar dependencia <code>jjwt 0.12.3</code> en <code>pom.xml</code> • Verificar que ambos proyectos corren: <code>localhost:3000</code> y <code>localhost:8080</code>
HORA 1	Backend: JWT + bcrypt + Endpoints	• Configurar <code>application.properties</code> (H2, JWT secret, CORS) • Crear entidades Usuario y Lead con JPA + Lombok • Implementar <code>JwtService</code> (<code>generateToken</code>, <code>extractEmail</code>, <code>isTokenValid</code>) • Implementar <code>AuthController</code>: POST <code>/auth/register</code> y POST <code>/auth/login</code> • Configurar Spring Security (CORS, CSRF disabled, filtro JWT) • Probar con <code>curl</code> o Postman los 6 endpoints
HORA 2	Frontend: Landing Page	• Crear componentes: Navbar, Hero, Services, Academy, Lab, Footer • Usar <code>v0.dev</code> o GitHub Copilot para generar diseño inicial • Implementar <code>ContactForm</code> que llama <code>POST /api/leads</code> • Verificar diseño responsive en móvil (DevTools F12)
HORA 3	Login Seguro + Dashboard	• Crear <code>/app/login/page.tsx</code> con validaciones de seguridad • Implementar Route Handler <code>/api/auth/set-cookie</code> (cookie <code>HttpOnly</code>) • Implementar Route Handler <code>/api/auth/logout</code> (elimina cookie) • Crear <code>middleware.ts</code> para proteger <code>/dashboard</code> • Crear <code>/app/dashboard/page.tsx</code> con RBAC (Admin/User)
HORA 3	Deploy + GitHub + Video	• <code>git init</code> && 3+ commits significativos && <code>git push</code> a GitHub • Deploy backend en Railway.app (conectar repo, detecta Spring Boot) • Deploy frontend en Vercel (conectar repo, agregar <code>NEXT_PUBLIC_API_URL</code>) • Grabar video de evidencia de 5 min mostrando los 6 puntos requeridos • Subir video a YouTube (no listado) o Google Drive público

PROMPTS RECOMENDADOS PARA IA GENERATIVA

Hero Section — v0.dev o Claude:

"Crea un componente Hero.tsx en Next.js 14 con Tailwind CSS para UQ AI Solution Company. Fondo negro (#0a0a1a) con gradiente azul/morado, título grande 'Inteligencia Artificial para el Perú y el Mundo', subtítulo, 2 botones CTA (Explorar Servicios, Contactar). Estilo oscuro moderno similar a deeplearning.ai."

Spring Security + JWT — GitHub Copilot:

"Configura Spring Security 6 para JWT stateless en Spring Boot 3.2. Endpoints públicos: /api/auth/**, POST /api/leads. Endpoint GET /api/leads solo ADMIN. CORS para http://localhost:3000 y *.vercel.app con allowCredentials=true. Explica cada línea de configuración."

Cookie HttpOnly — Claude:

"En Next.js 14 App Router, crea un Route Handler POST /api/auth/set-cookie que recibe { token, rol } y guarda el JWT en una cookie con: httpOnly=true, secure=true (producción), sameSite='strict', maxAge=86400. Explica por qué NO debemos usar localStorage para guardar JWT."

Dashboard con RBAC — ChatGPT:

"Crea una Server Component page.tsx en /app/dashboard para Next.js 14 que: (1) lee el token JWT de las cookies del servidor, (2) si no hay token redirige a /login, (3) si el rol es ADMIN muestra tabla de leads consumiendo GET /api/leads con el JWT en el header Authorization, (4) si el rol es USER solo muestra su propio perfil. Usar Tailwind CSS con fondo oscuro."

CONCEPTOS DE PROGRAMACIÓN SEGURA APLICADOS EN EL PROYECTO

CONCEPTO DEL SÍLABO	APLICACIÓN EN EL PROYECTO
bcrypt (S2)	BCryptPasswordEncoder.encode() en registro. Nunca texto plano en BD.
Cookie HttpOnly (S3)	JWT en cookie HttpOnly: document.cookie no puede leerla desde JavaScript.
Cookie SameSite=Strict (S3)	La cookie no se envía en requests cross-site → protección CSRF.
HTTPS / Secure (S2)	Vercel provee TLS automático. Atributo Secure=true en producción.
RBAC (S3)	hasRole("ADMIN") en Spring Security. Dashboard diferenciado por rol.
Query parametrizada (S2)	JPA/Hibernate usa queries parametrizadas por defecto → sin SQL Injection.
Mensaje genérico (S3)	"Credenciales incorrectas" — no revela si el email existe en el sistema.
CORS configurado (S2)	Solo los dominios del frontend autorizados pueden llamar al backend.
Validación de inputs (S2)	@NotBlank, @Email, @Valid en DTOs del backend.
Rate limiting cliente	Bloqueo 30s tras 3 intentos fallidos en el botón de login.
OWASP A01 (S1)	Middleware Next.js + Spring Security protegen rutas y endpoints.
OWASP A02 (S1)	bcrypt rounds=12, JWT con clave secreta fuerte en variables de entorno.
OWASP A07 (S1)	JWT con expiración, logout elimina cookie en el servidor.

ENTREGABLES

#	ENTREGABLE	FORMATO
1	URL del sitio desplegado en Vercel (dominio .vercel.app)	Link público
2	URL del repositorio GitHub (público, mínimo 3 commits)	Link GitHub
3	Screenshots del panel H2 con usuarios y leads registrados	Capturas de pantalla
4	Screenshots de Postman mostrando los 6 endpoints funcionando	Capturas de pantalla
5	Video de evidencia (máx. 5 min, link público)	YouTube / Drive / Loom

Evaluación Parcial · Programación Segura DD281 · Universidad Autónoma del Perú

Docente: Mg. Ruben Quispe Llachtarimay · Semestre 2026-1



CÓDIGO GUÍA — IMPLEMENTACIÓN PASO A PASO

📌 Este código es una GUÍA de referencia. Debes adaptarlo a tu proyecto, entenderlo y poder explicarlo. No copies y pegues sin leer — el docente puede preguntar sobre cualquier línea.

PASO 1 — Crear proyecto Next.js (Terminal)

```
# Crear proyecto Next.js 14 con TypeScript + Tailwind CSS
npx create-next-app@14 uq-ai-frontend \
  --typescript --tailwind --eslint --app --src-dir

cd uq-ai-frontend

# Instalar dependencias adicionales
npm install axios lucide-react

# Verificar que corre
npm run dev      # Abrir http://localhost:3000
```

PASO 2 — Crear proyecto Spring Boot

1. Ir a <https://start.spring.io/> y seleccionar:

- Project: Maven | Language: Java | Spring Boot: 3.2.x
- Group: com.uqai | Artifact: backend | Name: T_Apellido_Nombre_backend
- Dependencies: Spring Web · Spring Security · Spring Data JPA · H2 Database · Lombok · Validation

2. Descargar, descomprimir y abrir en IntelliJ IDEA o VS Code.

3. Agregar en pom.xml (dentro de <dependencies>):

```
<!-- jwt para JWT en Spring Boot 3 -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.3</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.3</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.3</version>
  <scope>runtime</scope>
</dependency>
```

PASO 3 — Configurar application.properties

Archivo: *src/main/resources/application.properties*

```
# Puerto del servidor
server.port=8080

# — H2 Database —
```

```

spring.datasource.url=jdbc:h2:mem:BD_apellido_nombre_T1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

# — JWT —————
# SEGURIDAD: en produccion mover a variable de entorno
jwt.secret=uqAiSolutionSecretKey2026ProgramacionSeguraDD281MustBe256bits
jwt.expiration=86400000

# — CORS —————
cors.allowed.origins=http://localhost:3000

```

PASO 4 — Entidades JPA

Archivo: *src/main/java/com/uqai/backend/entity/Usuario.java*

```

package com.uqai.backend.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;

@Entity
@Table(name = "usuarios")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class Usuario {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    @NotBlank private String nombre;

    @Column(nullable = false)
    @NotBlank private String apellidos;

    @Column(unique = true, nullable = false)
    @Email private String email;

    @Column(nullable = false)
    private String password; // SIEMPRE hash bcrypt - NUNCA texto plano

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Rol rol; // enum: ADMIN, USER

    @Column(nullable = false)
    @NotBlank private String area;
}

```

Archivo: *src/main/java/com/uqai/backend/entity/Lead.java*

```

package com.uqai.backend.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
import org.hibernate.annotations.CreationTimestamp;
import java.time.LocalDateTime;

@Entity @Table(name="leads")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class Lead {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank private String nombre;
    @Email private String email;
    private String empresa;
    private String telefono;

    @Column(length = 1000)
    private String mensaje;

    @CreationTimestamp
    private LocalDateTime fechaRegistro; // Se asigna automaticamente al guardar
}

```

PASO 5 — Servicio JWT

 **PROGRAMACIÓN SEGURA:** El JWT se firma con HMAC-SHA256 usando una clave secreta larga (256 bits). Nunca envíes el JWT sin firmar. El cliente NO puede modificar el payload sin invalidar la firma.

Archivo: ***src/main/java/com/uqai/backend/security/JwtService.java***

```

package com.uqai.backend.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.expiration}")
    private long expiration;

    // Generar token JWT firmado con HS256
    public String generateToken(String email) {
        return Jwts.builder()
            .subject(email)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expiration))

```

```

        .signWith(getSigningKey())
        .compact();
    }

    // Extraer el email (subject) del token
    public String extractEmail(String token) {
        return Jwts.parser().verifyWith(getSigningKey())
            .build().parseSignedClaims(token)
            .getPayload().getSubject();
    }

    // Validar token: firma correcta + no expirado + email coincide
    public boolean isTokenValid(String token, UserDetails userDetails) {
        try {
            return extractEmail(token).equals(userDetails.getUsername())
                && !isTokenExpired(token);
        } catch (JwtException e) {
            return false; // Token malformado, expirado o firma invalida
        }
    }

    private boolean isTokenExpired(String token) {
        return Jwts.parser().verifyWith(getSigningKey())
            .build().parseSignedClaims(token)
            .getPayload().getExpiration().before(new Date());
    }

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(secretKey.getBytes(StandardCharsets.UTF_8));
    }
}

```

PASO 6 — Configuración Spring Security

 **PROGRAMACIÓN SEGURA:** JWT es stateless — no hay sesión en el servidor. CSRF desactivado porque con JWT + SameSite la protección la da la cookie. CORS configurado explícitamente — no usar @CrossOrigin en cada controller.

Archivo: ***src/main/java/com/uqai/backend/security/SecurityConfig.java***

```

@Configuration @EnableWebSecurity @RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable) // JWT stateless: sin CSRF de formulario
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .sessionManagement(s ->
                s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                // Publicos: registro, login, H2 console
                .requestMatchers("/api/auth/**", "/h2-console/**").permitAll()
                // Cualquiera puede enviar un lead (formulario de contacto)
                .requestMatchers(HttpMethod.POST, "/api/leads").permitAll()
                // Solo ADMIN puede ver todos los usuarios o todos los leads
            );
    }
}

```

```

        .requestMatchers("/api/usuarios", "/api/leads").hasRole("ADMIN")
        // Todo lo demas requiere autenticacion
        .anyRequest().authenticated()
    )
    .headers(h -> h.frameOptions(f -> f.disable())) // Para H2 console
    .addFilterBefore(jwtAuthFilter,
        UsernamePasswordAuthenticationFilter.class)
    .build();
}

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    // Dominios permitidos: local + Vercel
    config.setAllowedOriginPatterns(
        List.of("http://localhost:3000", "https://*.vercel.app"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true); // Necesario para cookies HttpOnly
    UrlBasedCorsConfigurationSource source =
        new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}

@Bean
public PasswordEncoder passwordEncoder() {
    // bcrypt con 12 rounds: seguro y con tiempo de hash aceptable (~300ms)
    return new BCryptPasswordEncoder(12);
}

@Bean
public AuthenticationManager authManager(
    AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}
}

```

PASO 7 — Controller de Autenticación + AuthService

```

// AuthController.java
@RestController @RequestMapping("/api/auth") @RequiredArgsConstructor
public class AuthController {
    private final AuthService authService;

    @PostMapping("/register")
    public ResponseEntity<UsuarioResponse> register(
        @Valid @RequestBody RegisterRequest req) {
        return ResponseEntity.ok(authService.register(req));
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@Valid @RequestBody LoginRequest req) {
        try {
            return ResponseEntity.ok(authService.login(req));
        } catch (BadCredentialsException e) {
            // SEGURIDAD: mensaje generico (no revela si el email existe)
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                .body(Map.of("error", "Credenciales incorrectas"));
        }
    }
}

```

```

    }
}

// AuthService.java - bcrypt aplicado
@Service @RequiredArgsConstructor
public class AuthService {
    private final UsuarioRepository usuarioRepo;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;
    private final AuthenticationManager authManager;

    public UsuarioResponse register(RegisterRequest req) {
        if (usuarioRepo.existsByEmail(req.getEmail()))
            throw new RuntimeException("Email ya registrado");

        Usuario u = Usuario.builder()
            .nombre(req.getNombre()) .apellidos(req.getApellidos())
            .email(req.getEmail())
            // PROGRAMACION SEGURA: hash bcrypt - nunca guardar texto plano
            .password(passwordEncoder.encode(req.getPassword()))
            .rol(Rol.USER) .area(req.getArea())
            .build();
        return UsuarioResponse.from(usuarioRepo.save(u));
    }

    public AuthResponse login(LoginRequest req) {
        // Spring Security verifica bcrypt automaticamente
        authManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                req.getEmail(), req.getPassword()));
        String token = jwtService.generateToken(req.getEmail());
        Usuario u = usuarioRepo.findByEmail(req.getEmail()).orElseThrow();
        return new AuthResponse(token, u.getRol().name(), "OK");
    }
}

```

PASO 8 — Verificar con curl o Postman

```

# 1. Registrar usuario ADMIN
curl -X POST http://localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d
'{"nombre":"Admin","apellidos":"UQ","email":"admin@uqai.pe","password":"Admin2026!","area":"Sistemas"}'

# 2. Login (guarda el token retornado)
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@uqai.pe","password":"Admin2026!"}'

# 3. Listar usuarios (reemplaza <TOKEN> con el valor recibido)
curl -H "Authorization: Bearer <TOKEN>" \
  http://localhost:8080/api/usuarios

# 4. Enviar lead (publico, sin token)
curl -X POST http://localhost:8080/api/leads \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Juan
Perez","email":"juan@empresa.pe","empresa":"MiEmpresa","mensaje":"Quiero info"}'

```

```
# 5. Ver leads (solo ADMIN)
curl -H "Authorization: Bearer <TOKEN>" \
http://localhost:8080/api/leads
```

✓ Si ves la lista de usuarios/leads en JSON, el backend está funcionando correctamente. Toma capturas de pantalla de Postman para incluirlas en tus entregables.

PASO 9 — Hero Section (components/Hero.tsx)

```
// components/Hero.tsx
export default function Hero() {
  return (
    <section className="relative min-h-screen bg-[#0a0a1a] flex items-center"
      style={{background:
        'radial-gradient(ellipse at 20% 50%, rgba(26,115,232,0.15) 0%,
transparent 60%), '
        + 'radial-gradient(ellipse at 80% 20%, rgba(103,58,183,0.12) 0%,
transparent 60%), '
        + '#0a0a1a'}}>
      <div className="max-w-7xl mx-auto px-6 py-32 text-center">

        {/* Badge superior */}
        <div className="inline-flex items-center gap-2 px-4 py-2 rounded-full
          bg-blue-900/30 border border-blue-700/50 mb-8">
          <span className="w-2 h-2 rounded-full bg-green-400 animate-pulse" />
          <span className="text-blue-300 text-sm">IA para el Perú y el Mundo</span>
        </div>

        {/* Título principal */}
        <h1 className="text-5xl md:text-7xl font-black text-white leading-tight mb-6">
          UQ AI{' '}
          <span className="bg-gradient-to-r from-blue-400 to-purple-400
            bg-clip-text text-transparent">
            Solution
          </span>
          <br /><Company>
        </h1>

        <p className="text-xl text-gray-400 max-w-2xl mx-auto mb-12">
          Transformamos empresas peruanas con Inteligencia Artificial:
          agentes, chatbots, automatización y formación especializada.
        </p>

        {/* Botones CTA */}
        <div className="flex flex-col sm:flex-row gap-4 justify-center">
          <a href="#servicios"
            className="px-8 py-4 bg-blue-600 hover:bg-blue-500 text-white
              font-bold rounded-xl transition-all text-lg">
            Explorar Servicios →
          </a>
          <a href="#contacto"
            className="px-8 py-4 border border-gray-600 hover:border-blue-400
              text-gray-300 hover:text-white rounded-xl transition-all text-
lg">
            Contactar
          </a>
        </div>
      </div>
    </section>
  );
}
```

PASO 10 — Página Login Segura (app/login/page.tsx)

🔒 PROGRAMACIÓN SEGURA: (1) Mensaje genérico para no revelar si el email existe. (2) Bloqueo de 30s tras 3 intentos — previene fuerza bruta. (3) El JWT se guarda en cookie HttpOnly, NO en localStorage.

```
"use client";
import { useState } from "react";
import { useRouter } from "next/navigation";
import axios from "axios";

export default function LoginPage() {
  const router = useRouter();
  const [form, setForm] = useState({ email: "", password: "" });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [intentos, setIntentos] = useState(0);
  const [bloqHasta, setBloqHasta] = useState(0);

  const bloqueado = Date.now() < bloqHasta;
  const segsBloq = Math.ceil((bloqHasta - Date.now()) / 1000);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (bloqueado) return;
    setLoading(true); setError("");
    try {
      const { data } = await axios.post(
        `${process.env.NEXT_PUBLIC_API_URL}/api/auth/login`, form,
        { withCredentials: true });

      // Guardar JWT en cookie HttpOnly (via Route Handler del servidor)
      // SEGURIDAD: JavaScript del navegador NO puede leer esta cookie
      await axios.post("/api/auth/set-cookie",
        { token: data.token, rol: data.rol });

      router.push("/dashboard");
    } catch {
      const nuevo = intentos + 1;
      setIntentos(nuevo);
      // SEGURIDAD: mensaje generico - no revelar si el email existe
      setError("Credenciales incorrectas. Verifica tu email y contraseña.");
      if (nuevo >= 3) {
        setBloqHasta(Date.now() + 30000); // bloquear 30 segundos
        setError("Demasiados intentos. Espera 30 segundos.");
        setIntentos(0);
      }
    } finally { setLoading(false); }
  };

  return (
    <div className="min-h-screen bg-gray-950 flex items-center justify-center">
      <div className="bg-gray-900 rounded-2xl p-8 w-full max-w-md
        border border-gray-800 shadow-2xl">
        <h1 className="text-2xl font-bold text-white text-center mb-2">
          UQ AI Solution
        </h1>
        <p className="text-gray-400 text-center mb-8">Panel de Administración</p>
        <form onSubmit={handleSubmit} className="space-y-4">
          <input type="email" placeholder="Email" required
            value={form.email}
            onChange={e => setForm({...form, email: e.target.value})}>

```

```

        className="w-full px-4 py-3 bg-gray-800 text-white rounded-lg
        border border-gray-700 focus:border-blue-500
        focus:outline-none" />
<input type="password" placeholder="Contraseña" required
    value={form.password}
    onChange={e => setForm({...form, password: e.target.value})}
    className="w-full px-4 py-3 bg-gray-800 text-white rounded-lg
    border border-gray-700 focus:border-blue-500
    focus:outline-none" />
{error && <p className="text-red-400 text-sm text-center">{error}</p>}
<button type="submit" disabled={loading || bloqueado}
    className="w-full py-3 bg-blue-600 hover:bg-blue-500 text-white
    font-bold rounded-lg transition-all disabled:opacity-50">
    {loading ? "Verificando..." :
    bloqueado ? `Bloqueado (${segsBlog}s)` : "Iniciar Sesión"}
</button>
</form>
</div>
</div>
);
}

```

PASO 11 — Route Handlers: Cookie HttpOnly + Logout

Archivo: *src/app/api/auth/set-cookie/route.ts*

```

// app/api/auth/set-cookie/route.ts
import { NextRequest, NextResponse } from "next/server";

export async function POST(req: NextRequest) {
    const { token, rol } = await req.json();
    const res = NextResponse.json({ ok: true });

    // PROGRAMACION SEGURA: JWT en cookie HttpOnly
    // JavaScript del navegador NO puede leer esta cookie con document.cookie
    // Esto previene robo de token via XSS
    res.cookies.set("auth_token", token, {
        httpOnly: true, // CRITICO: JS no puede acceder
        secure: process.env.NODE_ENV === "production", // Solo HTTPS en prod
        sameSite: "strict", // CSRF: cookie no viaja en requests cross-site
        maxAge: 86400, // 24 horas en segundos
        path: "/",
    });

    // El rol puede ser leído por JS para mostrar UI condicional (no es secreto)
    res.cookies.set("user_rol", rol, {
        httpOnly: false,
        sameSite: "strict",
        maxAge: 86400,
        path: "/",
    });

    return res;
}

```

Archivo: *src/app/api/auth/logout/route.ts*

```
// app/api/auth/logout/route.ts
import { NextResponse } from "next/server";

export async function POST(req: Request) {
  const res = NextResponse.redirect(
    new URL("/login", req.url));

  // PROGRAMACION SEGURA: eliminar cookie en el servidor (Max-Age=0)
  // No basta con limpiar el estado en el cliente
  res.cookies.set("auth_token", "", { maxAge: 0, path: "/" });
  res.cookies.set("user_rol", "", { maxAge: 0, path: "/" });

  return res;
}
```

PASO 12 — Middleware de Rutas Protegidas (middleware.ts)

Archivo en la raíz del proyecto: *middleware.ts*

```
// middleware.ts ← en la raíz del proyecto, mismo nivel que package.json
import { NextRequest, NextResponse } from "next/server";

export function middleware(req: NextRequest) {
  const token = req.cookies.get("auth_token")?.value;
  const pathname = req.nextUrl.pathname;

  // Rutas que requieren autenticacion
  const protected_routes = ["/dashboard", "/admin"];
  const isProtected = protected_routes.some(r => pathname.startsWith(r));

  // Sin token → redirigir a login
  if (isProtected && !token) {
    const url = new URL("/login", req.url);
    url.searchParams.set("redirect", pathname);
    return NextResponse.redirect(url);
  }

  // Ya autenticado → no dejar entrar a /login de nuevo
  if (pathname === "/login" && token) {
    return NextResponse.redirect(new URL("/dashboard", req.url));
  }

  return NextResponse.next();
}

export const config = {
  // Rutas donde aplica el middleware (excluir assets)
  matcher: ["/dashboard/:path*", "/admin/:path*", "/login"],
};
```

PASO 13 — Dashboard con RBAC (app/dashboard/page.tsx)

```
// app/dashboard/page.tsx — Server Component
import { cookies } from "next/headers";
import { redirect } from "next/navigation";

async function getLeads(token: string) {
  try {
    const res = await fetch(
```

```

    `${process.env.NEXT_PUBLIC_API_URL}/api/leads`,
    { headers: { Authorization: `Bearer ${token}` }, cache: "no-store" });
    if (!res.ok) return [];
    return res.json();
  } catch { return []; }
}

export default async function DashboardPage() {
  const cookieStore = cookies();
  const token = cookieStore.get("auth_token")?.value;
  const rol = cookieStore.get("user_rol")?.value;

  // Doble verificación (middleware ya redirige, pero por si acaso)
  if (!token) redirect("/login");

  // RBAC: solo ADMIN puede ver todos los leads
  const leads = rol === "ADMIN" ? await getLeads(token) : [];

  return (
    <div className="min-h-screen bg-gray-950 p-8">
      <div className="max-w-6xl mx-auto">
        <div className="flex justify-between items-center mb-8">
          <h1 className="text-3xl font-bold text-white">
            Panel {rol === "ADMIN" ? "🔥 Administrador" : "👤 Usuario"}
          </h1>
          <form action="/api/auth/logout" method="POST">
            <button className="px-4 py-2 bg-red-700 hover:bg-red-600
              text-white rounded-lg">
              Cerrar Sesión
            </button>
          </form>
        </div>

        {rol === "ADMIN" && (
          <div className="bg-gray-900 rounded-xl border border-gray-800">
            <div className="p-4 border-b border-gray-800">
              <h2 className="text-xl text-white font-semibold">
                Leads recibidos ({leads.length})
              </h2>
            </div>
            <div className="overflow-x-auto">
              <table className="w-full text-sm text-gray-300">
                <thead className="bg-gray-800 text-xs uppercase text-gray-400">
                  <tr>
                    {["Nombre", "Email", "Empresa", "Mensaje", "Fecha"].map(h => (
                      <th key={h} className="px-4 py-3 text-left">{h}</th>
                    ))}
                  </tr>
                </thead>
                <tbody className="divide-y divide-gray-800">
                  {leads.map((l: any) => (
                    <tr key={l.id} className="hover:bg-gray-800">
                      <td className="px-4 py-3">{l.nombre}</td>
                      <td className="px-4 py-3">{l.email}</td>
                      <td className="px-4 py-3">{l.empresa ?? "—"}</td>
                      <td className="px-4 py-3 max-w-xs truncate">{l.mensaje}</td>
                      <td className="px-4 py-3 text-gray-500">
                        {new Date(l.fechaRegistro).toLocaleDateString("es-PE")}
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          </div>
        )}
      </div>
    </div>
  );
}

```

```

        }}}
    </tbody>
</table>
</div>
</div>
)}

{rol !== "ADMIN" && (
    <div className="bg-gray-900 rounded-xl p-6 border border-gray-800">
        <p className="text-gray-400">Eres usuario con rol
            <span className="text-blue-400 font-bold mx-1">USER</span>.
            Contacta al administrador para ampliar permisos.
        </p>
    </div>
    )}
</div>
</div>
);
}

```

DEPLOY — PASO A PASO

PASO 14 — Variables de Entorno

Frontend — archivo **.env.local** (en la raíz del proyecto Next.js):

```
NEXT_PUBLIC_API_URL=http://localhost:8080
```

En el dashboard de Vercel, agregar la variable (apuntando al backend desplegado):

```
NEXT_PUBLIC_API_URL=https://tu-backend.up.railway.app
```

PASO 15 — Deploy Backend en Railway.app

7. Crear cuenta gratuita en <https://railway.app>
8. Clic en "New Project" → "Deploy from GitHub repo"
9. Seleccionar el repositorio del backend (Spring Boot)
10. Railway detecta automáticamente el proyecto Maven
11. En Settings → agregar variable de entorno: PORT=8080
12. Copiar la URL pública generada: <https://tu-proyecto.up.railway.app>
13. Verificar: <https://tu-proyecto.up.railway.app/api/auth/login> (debe responder 400 o 405)

PASO 16 — Deploy Frontend en Vercel

```
# Opcion A: desde terminal (Vercel CLI)
npm install -g vercel
vercel login
vercel          # Sigue las instrucciones interactivas

# Opcion B: desde vercel.com (mas facil)
# 1. Ir a https://vercel.com → New Project
# 2. Conectar repositorio GitHub del frontend
# 3. Framework: Next.js (deteccion automatica)
# 4. Environment Variables → NEXT_PUBLIC_API_URL = https://tu-backend.up.railway.app
# 5. Clic Deploy → obtener URL publica ✅
```

PASO 17 — GitHub: Commits Significativos

```
# En la raíz del proyecto
git init

# COMMIT 1 — Setup inicial
git add .
git commit -m "feat: inicializar Next.js + Spring Boot con estructura base DD281"

# COMMIT 2 — Backend completo
git add backend/
git commit -m "feat: implementar API REST con JWT + bcrypt + H2 (OWASP A02/A07)"

# COMMIT 3 — Frontend landing + login seguro
git add frontend/
git commit -m "feat: landing UQ AI + login seguro cookie HttpOnly + RBAC dashboard"

# COMMIT 4 — Deploy config
git add .
git commit -m "chore: configurar env vars para deploy Vercel + Railway"
```

```
# Subir a GitHub
git remote add origin https://github.com/TU_USUARIO/uqai-solution-web.git
git branch -M main
git push -u origin main
```

✅ RESULTADO FINAL: Tu sitio UQ AI Solution estará disponible en una URL pública de Vercel (ej: <https://uq-ai-solution.vercel.app>) con HTTPS automático, backend en Railway y todo el código en GitHub con historial de commits.

LINKS Y RECURSOS RECOMENDADOS

DOCUMENTACIÓN OFICIAL

Next.js 14 App Router	https://nextjs.org/docs/app
Tailwind CSS	https://tailwindcss.com/docs
Spring Boot 3	https://spring.io/projects/spring-boot
Spring Security	https://docs.spring.io/spring-security/reference/
jjwt (JWT en Java)	https://github.com/jwt/jjwt#readme
H2 Database	https://www.h2database.com/html/main.html

TUTORIALES VIDEO

Spring Boot + JWT completo (YouTube)	https://youtu.be/BVdQ3iuovg0
Next.js 14 Auth con cookies HttpOnly	https://youtu.be/DJvM2lSPn6w
Deploy Spring Boot en Railway	https://docs.railway.app/guides/spring-boot
Deploy Next.js en Vercel	https://vercel.com/docs/frameworks/nextjs

HERRAMIENTAS IA PARA CÓDIGO

GitHub Copilot (VS Code / IntelliJ)	https://github.com/features/copilot
Claude — generación de componentes	https://claude.ai
v0 by Vercel — UI desde descripción	https://v0.dev
ChatGPT — consultas y debugging	https://chat.openai.com
Bolt.new — prototipo full-stack rápido	https://bolt.new

SITIOS DE REFERENCIA VISUAL

deeplearning.ai — referencia de diseño	https://www.deeplearning.ai/
landing.ai — referencia de diseño	https://landing.ai/
Dribbble — inspiración UI dark mode	https://dribbble.com

CHECKLIST FINAL ANTES DE ENTREGAR

BACKEND

- ☐ Spring Boot corre en localhost:8080 sin errores
- ☐ Consola H2 accesible en /h2-console con BD: BD_apellido_nombre_T1
- ☐ POST /api/auth/register → registra usuario con password hashado (bcrypt)
- ☐ POST /api/auth/login → retorna JWT válido
- ☐ GET /api/usuarios → solo accesible con JWT de usuario ADMIN
- ☐ POST /api/leads → guarda lead sin necesitar token (público)
- ☐ GET /api/leads → solo accesible con JWT de ADMIN
- ☐ Capturas de Postman de todos los endpoints

FRONTEND

- ☐ Landing page carga en localhost:3000 con Hero visible
- ☐ Navbar responsivo (probar en DevTools F12 → mobile)

- [] 3 secciones: UQ AI Solutions + Academy + Lab
- [] Formulario de contacto envía lead al backend (verificar en H2)
- [] Footer completo con información de la empresa

LOGIN SEGURO

- [] Página /login con formulario Email + Contraseña
- [] Mensaje de error GENÉRICO (no revela si el email existe)
- [] Botón se bloquea 30s tras 3 intentos fallidos
- [] JWT guardado en cookie HttpOnly (verificar en DevTools > Application > Cookies)
- [] Acceder a /dashboard sin login → redirige a /login
- [] Admin ve tabla de leads — Usuario USER solo ve su perfil
- [] Logout elimina la cookie (verificar en DevTools que desaparece)

DEPLOY Y ENTREGABLES

- [] Repositorio GitHub público con mínimo 3 commits significativos
- [] Backend desplegado en Railway.app o Render.com (URL pública)
- [] Frontend desplegado en Vercel (URL pública .vercel.app)
- [] Video de 5 min grabado mostrando los 6 puntos de evidencia
- [] Link del video público (YouTube / Drive / Loom)